



Kotlin Coroutines: An Introduction to Asynchronous Programming in Android.

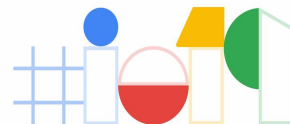
Jabez Magomere
(Android Developer)

 @mc_jayb



<https://github.com/JabezNzomo99>

Repository: <https://github.com/JabezNzomo99/GoogleIOExtended19>





Introduction to Coroutines

The concept of coroutines has existed historically in other programming languages such as python, C#, Ruby, Go and JavaScript.

Coroutines is a powerful concept that allows us to achieve asynchronous programming. The world is not sequential, multiple events happen at the same time, e.g. systems communicating, hence the need for concurrency.

Coroutines are light weight threads launched with a coroutine builder in a context of some sort referred to as a coroutine scope. The lifetime of the coroutine is limited to the scope in which it is launched.

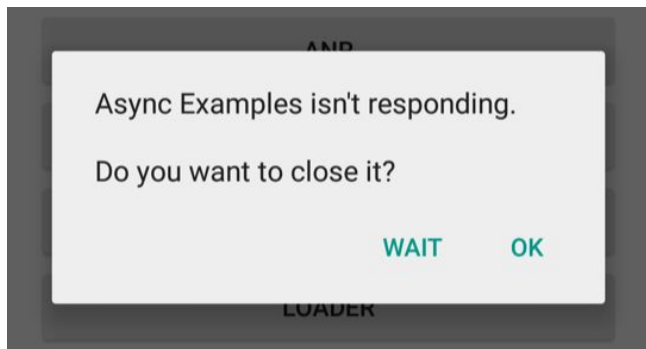


Don't block Keep moving

A Tasks  Threads

What problems does coroutines solve in Android?

Long Running Tasks- Performing long operations such as network access or database queries will block the UI thread, yielding poor performance. When the thread is blocked, no events can be dispatched including drawing events. From the user's perspective the application appears to hang. The infamous ANR.





Suspending Functions

- Coroutines are based on suspending functions.
- **Suspend** keyword is used to mark functions that perform blocking operations. e.g network requests, writing to a file or performing a resource intense computation.
- Suspend functions may suspend the execution of the current coroutine without blocking the main thread. Suspend keyword allows us to write asynchronous code sequentially.
- Suspend functions can only run inside another suspend function or a coroutine.
- Suspend functions do not have any special return types such as; Future or Promise.




```
/**
```

```
Marking this function as suspend implies that it may be suspended and  
later on resumed because it performs a long running task
```

```
*/
```

```
suspend function fetchAllUsers():List<User>{  
    return userRepository.fetchAllusers()  
}
```



Suspend and Resume

Coroutines build upon regular functions by adding two new operations. In addition to `invoke`(or `call`) and `return`, coroutines add **suspend** and **resume**. Suspend and Resume work together to replace callbacks.

Suspend- pause the execution of the current coroutine, saving all local variables.

Resume- continue a suspended function from the place it was paused.

When all coroutines are suspended the main thread is free to do other work.



```
// Dispatchers.Main
suspend fun fetchDocs() {
    // Dispatchers.IO
    val result = get("developer.android.com")
    // Dispatchers.Main
    show(result)
}

suspend fun get(url: String) = withContext(Dispatchers.IO){/*...*/}
```



```
suspend fun fetchDocs() {  
    val docs = get("...")  
    show(docs)  
}
```

suspend

resume

Main Thread
[stack]



Coroutine Builders

Coroutine builders are functions that create a new coroutine to run a suspending function.

Examples of coroutine builders: `runBlocking`, `launch`, `async`.



runBlocking

Blocks the current thread until all the tasks inside that coroutine are finished.

```
fun main() {  
    println("Hello,")  
    //block the main thread while waiting for the coroutine to finish its execution  
    runBlocking {  
        delay(2000L) // suspends the current coroutine for 2 seconds  
    }  
    // will be executed after 2 seconds  
    println("World!")  
}
```



launch

Launches a new coroutine without blocking the current thread and returns a reference to the coroutine as a job. The coroutine is cancelled when the job is cancelled. Also referred to as “fire and forget”- it won't return the result to the caller.

```
//Under the hood
fun CoroutineScope.launch(
    context: CoroutineContext = EmptyCoroutineContext,
    start: CoroutineStart = CoroutineStart.DEFAULT,
    block: suspend CoroutineScope.() -> Unit
): Job { source }
```



launch continued ...

The builder always needs a scope (To be discussed ahead).



```
GlobalScope.launch(Dispatchers.Main) {  
    ...  
}
```



async

Creates a coroutine and runs its future result as an implementation of deferred similar to *Future* or *Promise*. The main difference between `async` and `launch` is that `async` returns a deferred object which you can `await` to get the result of the operation unlike `launch` which returns a job object. `Async` does not throw exceptions by default since it will return a result or exception to the caller.

```
GlobalScope.launch(Dispatchers.Main) {  
  
    val currentFriends = async(Dispatchers.IO) { userService.requestCurrentFriends(user) }  
    val suggestedFriends = async(Dispatchers.IO) { userService.requestSuggestedFriends(user) }  
  
    val allFriends = currentFriends.await()+suggestedFriends.await()  
  
}
```



Coroutine Context

Coroutines always execute in some context which is represented by the value of **CoroutineContext** type which contains a **job** and a **dispatcher** amongst other elements.

It can be said to be a set of rules and configurations that define how the coroutine will be executed.

A coroutine builder (launch, async) receives a coroutine context to determine how to configure the coroutines.

All coroutine builders like launch and async accept an optional CoroutineContext that can be used to specify the dispatcher for the new coroutine among other elements.



Dispatchers

A coroutine dispatcher determines what thread a corresponding coroutine uses for its execution. There are three main types of dispatchers: **Dispatchers.Main**, **Dispatchers.IO**, **Dispatchers.Default**.

<code>Dispatchers.<u>Main</u></code>	<code>Dispatchers.<u>IO</u></code>	<code>Dispatchers.<u>Default</u></code>
Main thread on Android, interact with the UI and perform light work	Optimized for disk and network IO off the main thread	Optimized for CPU intensive work off the main thread
- Calling suspend functions - Call UI functions - Updating LiveData	- Database* - Reading/writing files - Networking**	- Sorting a list - Parsing JSON - DiffUtils



```
fun insertUser(user:User):Unit = launch(Dispatchers.IO){  
    userRepository.insertUser(user)  
}
```



withContext

This is a function that allows to easily change the context that will be used to run a part of the code inside a coroutine. The section of code wrapped inside withContext block will be run on the specified dispatcher. Allows you to specify what thread executes what block of code. It ensures that every function is safe to be called on any Dispatcher, including main.

```
suspend fun get(url: String) =  
    // Dispatchers.IO  
    withContext(Dispatchers.IO) {  
        // Dispatchers.IO  
        /* perform blocking network IO here */  
    }  
    // Dispatchers.Main
```



Structured Concurrency

Keeping track of all coroutines manually can be a tiresome task. Poor code can result to a work leak.

A leaked coroutine can waste memory , CPU disk or even launch a request that's not needed. Structured concurrency ideally helps to prevent coroutines from leaking.

Structured concurrency is a combination of language features and best practices that when followed, help you keep track of all work running in coroutines.

Goals of Structured Concurrency

- Cancel work that is no longer needed.
- Keep track of work while it's running.
- Signal errors when a coroutine fails.



Guarantees of Structured Concurrency

1. When a scope cancels, all of its coroutines cancel.
2. When a suspend fun returns, all of its work is done.
3. When a coroutine errors, its caller or scope is notified.



Coroutine Scope

- Defines a new scope for coroutines. In Android, lifecycle changes are very important.
- Background tasks should not try to update the UI when the UI is destroyed because it will throw an exception.
- Scopes help us to launch coroutines on entities with a well-defined lifecycle. E.g. Viewmodel, Activity or Fragment.
- A coroutine scope keeps track of all the coroutines started in it and it can cancel the coroutines started in it. In android, it makes sense to associate a coroutine scope with a user view. Kotlin does not allow you to start a new coroutine without a coroutine scope.



Global Scope

It's a general scope that can be used for any coroutines that are meant to continue executing while the App is running. Coroutines launched in this scope are not tied to any components that can be destroyed. However, it is advisable not to use the global scope in android applications.

```
GlobalScope.launch(Dispatchers.Main) {  
  
    /**  
    perform long running operations, e.g fetch data from remote source  
    **/  
  
}
```



ViewModel Scope

```
package com.jabezmagomere.movies.ui

import androidx.lifecycle.ViewModel
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.SupervisorJob
import kotlin.coroutines.CoroutineContext

abstract class BaseViewModel:ViewModel(),CoroutineScope{
    private val job = Job()
    override val coroutineContext: CoroutineContext
        get() = (Dispatchers.Main + job)

    override fun onCleared() {
        super.onCleared()
        job.cancel()
    }
}
```

```
package com.jabezmagomere.movies.ui.viewmodel

import androidx.annotation.UiThread
import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel
import com.jabezmagomere.movies.data.models.Movie
import com.jabezmagomere.movies.data.repository.MovieRepository
import com.jabezmagomere.movies.ui.BaseViewModel
import com.jabezmagomere.movies.ui.view.Category
import com.jabezmagomere.movies.util.lazyDeferred
import kotlinx.coroutines.*
import retrofit2.HttpException
import java.lang.Exception
import kotlin.coroutines.CoroutineContext

class MainAcitivityViewModel(private val movieRepository: MovieRepository):BaseViewModel(){

    val trendingMoviesToday = MutableLiveData<List<Movie>>()
    val trendingMoviesThisWeek = MutableLiveData<List<Movie>>()
    val allCategory = MutableLiveData<ArrayList<Category>>()
    val exception = MutableLiveData<Exception>()

    fun getTrendingMoviesToday()=launch {
        withContext(Dispatchers.IO){
            try {
                val movies = movieRepository.getTrendingMoviesToday()
                trendingMoviesToday.postValue(movies)
            }catch (ex:Exception){
                exception.postValue(ex)
            }
        }
    }

    fun getTrendingMoviesThisWeek()= launch {
        withContext(Dispatchers.IO){
            try {
                trendingMoviesThisWeek.postValue(movieRepository.getTrendingMoviesThisWeek())
            }catch (ex:Exception){
                exception.postValue(ex)
            }
        }
    }
}
```



What have i been saying all along?

Coroutines are light weight threads that help achieve asynchronous programming.

Coroutines leverage upon suspending functions to **suspend** long operations and **resume** later allowing the current thread to perform other tasks.

A **suspend function** can only be called in another suspend function or coroutine.

Coroutine builders are used to launch coroutines.

Dispatchers are threads that execute your coroutines.

Coroutine scopes are used to keep track of all coroutines to ensure structured concurrency.

Deep dive into code

